



P.E.S. College of Engineering, Mandya-571401
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)



Department of Information Science and Engineering

Project Phase 3 presentation on :

“AI-Based Dynamic Two-Handed Sign Language Recognition System”

Under the Guidance of :

Dr Rakshith N

Associate Professor Information
Science & Engineering

Team members:

- Chinmayee B
- Dhanush Patel M
- Meghana G K
- Suman Kumar Matho

Sl.no	Content Title:
1	Introduction
2	Problem Statement
3	Objectives 1 Methodology
4	Output
5	Objectives 2 Methodology
6	Output
	Objectives 3 Methodology
7	Output
8	Tools and Languages Used
9	References

Introduction

- **Sign language is an important communication method used by people with hearing and speech impairments**, but many people are not familiar with it, which creates a communication barrier.
- **Indian Sign Language (ISL) recognition systems aim to bridge this gap** by automatically translating hand gestures into understandable text or speech using computer vision techniques.
- **Advancements in artificial intelligence and deep learning**, especially object detection models like YOLO, enable accurate detection and recognition of hand gestures from images and videos.
- This project focuses on developing a **real-time system** that recognizes dynamic **two-handed sign language gestures** and converts them into meaningful text and speech through a web-based application.

Contribution to SDG 9: Industry, Innovation, and Infrastructure

The project uses advanced AI technologies such as computer vision and YOLO to develop an innovative communication system.

SDG 10 – Reduced Inequalities

The system helps reduce communication barriers between hearing-impaired individuals and others, promoting equal opportunities and inclusive communication in society.

Abstract

- The model uses **YOLOv8 classification architecture** for learning visual features and recognizing dynamic two-handed Indian Sign Language gestures.
- The system is developed using the **ISL-CSLTR dataset**, which contains annotated sentence-level sign language frames representing multiple gesture classes.
- The dataset is organized into **training and validation sets** to ensure effective model training and performance evaluation.
- The trained model is evaluated using metrics such as **accuracy, precision, recall, and F1-score**, and training performance is analyzed using loss and accuracy graphs.
- A **web-based application interface** is designed to allow users to upload gesture images or perform sign gestures and receive predicted sign language outputs.

Problem Statement



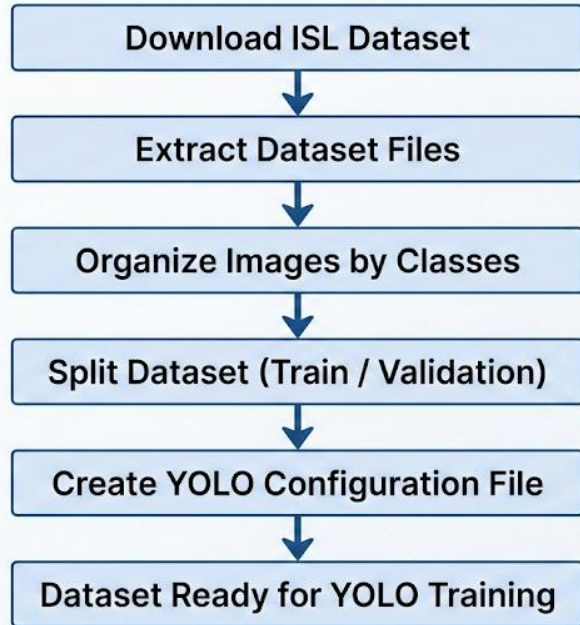
Communication between **hearing-impaired individuals** and **normal speakers** is difficult because most people do not understand **Indian Sign Language (ISL)**. Existing **sign language recognition systems** recognize only **limited gestures** and often lack **real-time translation** capability. Therefore, an efficient system is needed to recognize **dynamic two-handed sign language gestures** and convert them into **text or speech** for better communication.

Project Objective

1. **To organize** the ISL dataset for training a YOLO-based model to recognize dynamic two-handed sign language gestures.
2. **To train** a YOLO-based model to recognize dynamic two-handed sign language gestures from the prepared dataset.
3. **To implement** a real-time sign language recognition system using the trained YOLOv8 model.
4. **To design** and implement a user-friendly web application interface that allows users to perform sign gestures and receive real-time translation output.

Objective 1

Methodology :



1. Download ISL-CSLTR Dataset

The ISL-CSLTR dataset is downloaded from the online repository using Google Colab.

2. Extract Dataset Files

The compressed dataset file is extracted to access the sign language frames and folders.

3. Organize Images by Classes

Images are grouped into folders based on sentence labels to represent different gesture classes.

4. Split Dataset (Train / Validation)

The dataset is divided into **training and validation sets** using an 80:20 ratio for model training and evaluation.

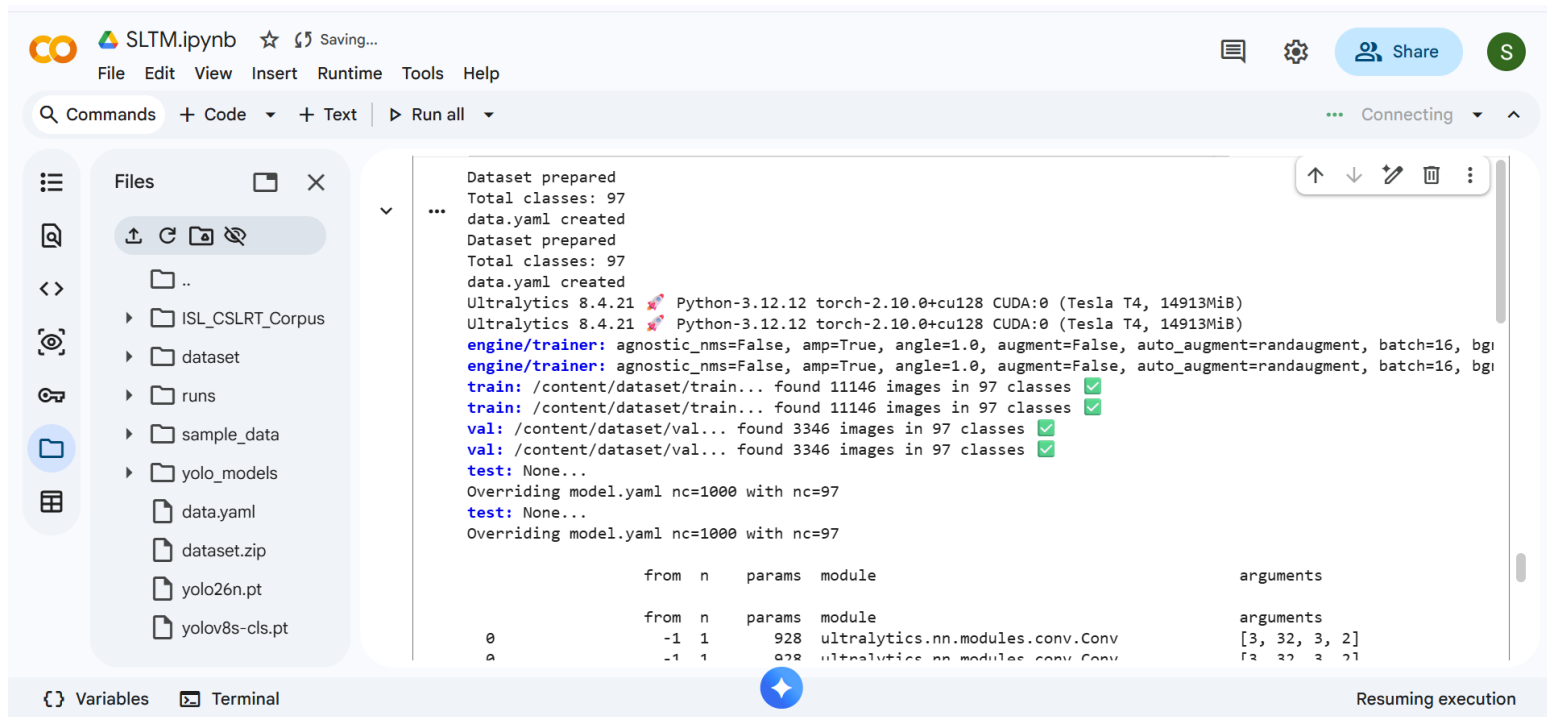
5. Create YOLO Configuration File

A **data.yaml** file is created to define dataset paths, number of classes, and class names for YOLO training.

6. Dataset Ready for YOLO Training

The prepared dataset is used to train the YOLO model for recognizing sign language gestures.

Output



The screenshot displays the SLTM.ipynb interface. The left sidebar shows a file explorer with a tree view containing folders like 'ISL_CSLRT_Corpus', 'dataset', 'runs', 'sample_data', and 'yolo_models', along with files 'data.yaml', 'dataset.zip', 'yolo26n.pt', and 'yolov8s-cls.pt'. The main area shows the execution output of a dataset preparation script. The output includes messages for dataset preparation, class counts, and file creation. It also lists system dependencies and training parameters. The bottom status bar indicates 'Resuming execution'.

```
Dataset prepared
Total classes: 97
data.yaml created
Dataset prepared
Total classes: 97
data.yaml created
Ultralytics 8.4.21 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Ultralytics 8.4.21 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr
train: /content/dataset/train... found 11146 images in 97 classes ✓
train: /content/dataset/train... found 11146 images in 97 classes ✓
val: /content/dataset/val... found 3346 images in 97 classes ✓
val: /content/dataset/val... found 3346 images in 97 classes ✓
test: None...
Overriding model.yaml nc=1000 with nc=97
test: None...
Overriding model.yaml nc=1000 with nc=97
```

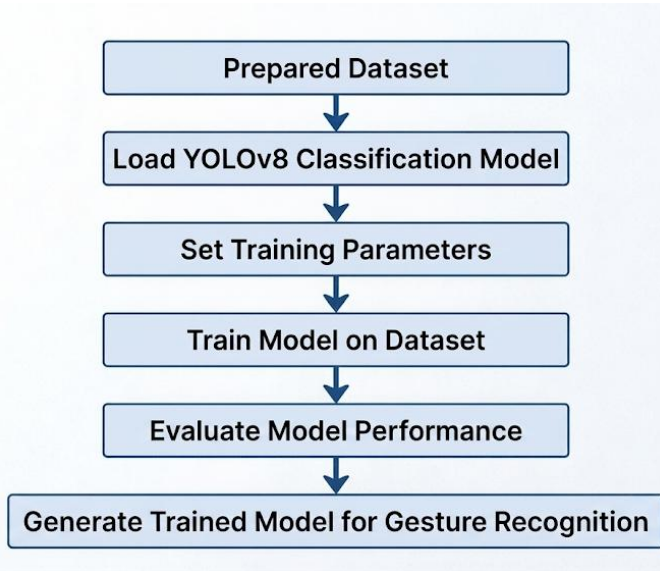
	from	n	params	module	arguments
	from	n	params	module	arguments
0	-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]
0	-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]

Resuming execution

Fig1 : Dataset preparation

Objective 2

Methodology :



1. Prepared Dataset

The organized ISL-CSLTR dataset containing training and validation images is provided as input to the model.

2. Load YOLOv8 Classification Model

The YOLOv8 classification architecture (`yolov8s-cls.pt`) is loaded using the Ultralytics framework in Google Colab.

3. Set Training Parameters

Training parameters such as dataset path, number of epochs, and image size are defined for the training process.

4. Train Model on Dataset

The YOLO model is trained using the training dataset, allowing it to learn visual features and patterns of different sign language gestures.

5. Evaluate Model Performance

The trained model is evaluated using validation data, generating metrics such as accuracy, precision, recall, and F1-score, along with training graphs and a confusion matrix.

6. Generate Trained Model

After successful training, the model is saved and can be used for predicting sign language gestures from new images.

Output



The screenshot shows a JupyterLab environment with the following components:

- File Explorer (Left):** Displays a file tree with folders like `ISL_CSLRT_Corp...`, `dataset`, `runs`, `sample_data`, and `yolo_models`. It also shows files like `data.yaml`, `dataset.zip`, `yolo26n.pt`, and `yolov8s-cls.pt`. The disk usage is shown as 50.37 GB available.
- Command Bar (Top):** Includes a search bar, tabs for `Commands`, `Code`, `Text`, and a `Run all` button.
- Code Editor (Main):** Contains a Python script snippet:

```
# 8 Download Model
# -----
files.download(zip_path)
```

The output of the script is displayed below the code:

```
Running 20 Test Cases...

Evaluation Metrics (20 Test Cases)

Accuracy : 85.0 %
Precision : 87.5 %
Recall : 85.0 %
F1 Score : 85.0 %

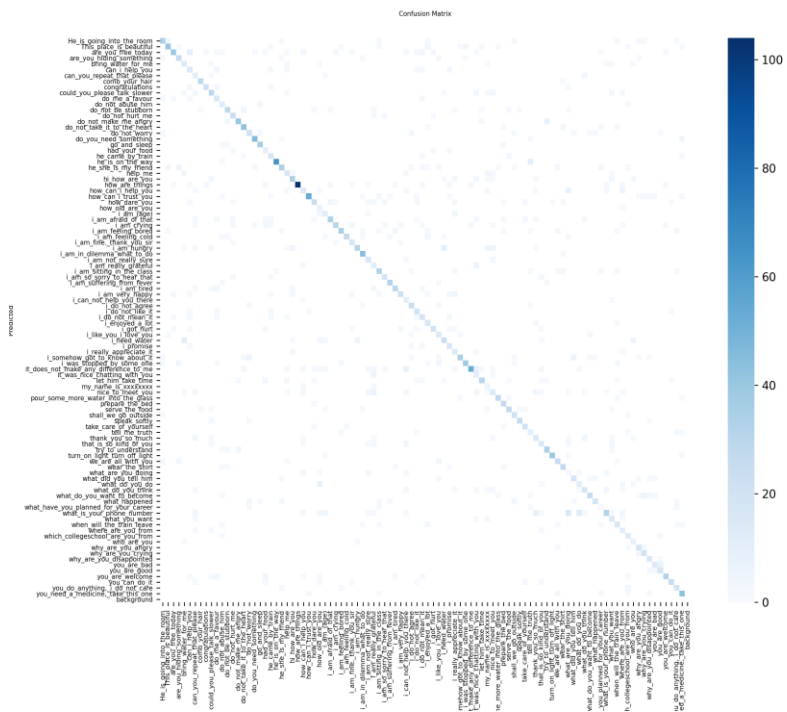
20 Test Case Results

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	Test Image	True Label	Predicted
0	/content/dataset/val/i_somewhat_got_to_know_abo...	i_somewhat_got_to_know_about_it	i_somewhat_got_to_know_at
4	/content/dataset/val/do_not_take_it_to_the_heart	do_not_take_it_to_the_heart	do_not_take_it_to_the
- Bottom Bar:** Shows `Variables`, `Terminal`, a status icon, and execution details: `Executing (2m 12s)` and `T4 (Python 3)`.

Fig2 : Evaluation metrics

Output



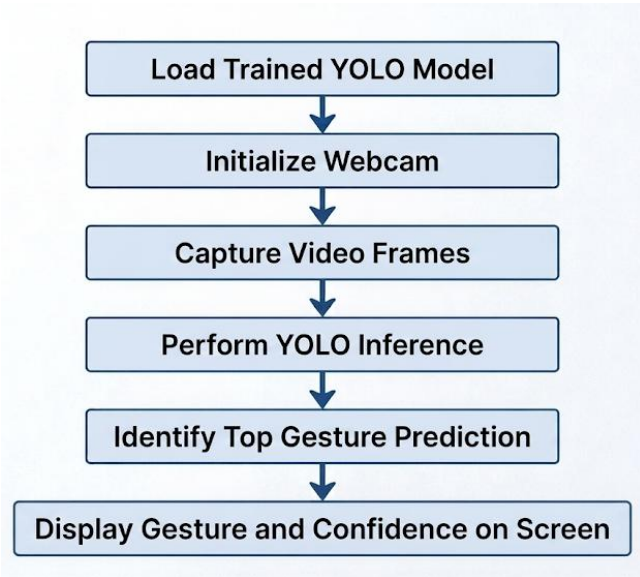
Run all

	Test Image	True Label	Predicted Label
0	/content/dataset/val/i_somewhat_got_to_know_abo...	i_somewhat_got_to_know_about_it	i_somewhat_got_to_know_about_it
1	/content/dataset/val/do_not_take_it_to_the_hea...	do_not_take_it_to_the_heart	do_not_take_it_to_the_heart
2	/content/dataset/val/how_can_i_trust_you/trust...	how_can_i_trust_you	how_can_i_trust_you
3	/content/dataset/val/do_not_take_it_to_the_hea...	do_not_take_it_to_the_heart	do_not_take_it_to_the_heart
4	/content/dataset/val/why_are_you_angry/angry (...)	why_are_you_angry	why_are_you_angry
5	/content/dataset/val/do_me_a_favour/favour (4)...	do_me_a_favour	can_i_help_you
6	/content/dataset/val/thank_you_so_much/thank y...	thank_you_so_much	thank_you_so_much
7	/content/dataset/val/you_do_anything_i_do_not...	you_do_anything_i_do_not_care	you_do_anything_i_do_not_care
8	/content/dataset/val/how_are_things/things (3)...	how_are_things	how_are_things
9	/content/dataset/val/it_was_nice_chatting_with...	it_was_nice_chatting_with_you	it_does_not_make_any_difference_to_me
10	/content/dataset/val/what_happened/happened (3)...	what_happened	what_happened
11	/content/dataset/val/you_need_a_medicine_take...	you_need_a_medicine_take_this_one	you_need_a_medicine_take_this_one
12	/content/dataset/val/take_care_of_yourself/tak...	take_care_of_yourself	i_am_afraid_of_that
13	/content/dataset/val/do_not_take_it_to_the_hea...	do_not_take_it_to_the_heart	do_not_take_it_to_the_heart
14	/content/dataset/val/i_am_afraid_of_that/afrai...	i_am_afraid_of_that	i_am_afraid_of_that
15	/content/dataset/val/do_me_a_favour/favour (4)...	do_me_a_favour	do_me_a_favour
16	/content/dataset/val/when_will_the_train_leave...	when_will_the_train_leave	when_will_the_train_leave
17	/content/dataset/val/comb_your_hair/comb (2) 1...	comb_your_hair	comb_your_hair
18	/content/dataset/val/what_do_you_do/what do yo...	what_do_you_do	what_do_you_do
19	/content/dataset/val/i_am_feeling_bored/i am f...	i_am_feeling_bored	i_am_feeling_bored

Fig2 : Confusion Matrix and sample test cases

Objective 3

Methodology :



1. Load Trained YOLO Model

The trained YOLOv8 classification model (best.pt) is loaded using the Ultralytics library to perform gesture recognition.

2. Initialize Webcam

The system accesses the computer webcam using OpenCV to capture real-time video input.

3. Capture Video Frames

The webcam continuously captures frames which act as input images for the recognition system.

4. Perform YOLO Inference

Each captured frame is passed to the YOLO model, which analyzes the image and predicts the gesture class.

5. Identify Top Gesture Prediction

The model selects the gesture with the highest probability and extracts the corresponding class label and confidence score.

6. Display Gesture and Confidence

The predicted gesture label and confidence value are displayed on the video frame, allowing real-time sign language recognition.

Output

```
Loading weights and class names from best.pt...  
Model loaded successfully! Loaded 97 classes.  
  
Starting camera... Press 'q' on the video window to quit.  
Recognized: try to understand (Confidence: 0.5792)
```

Fig 4 : the system successfully detects and recognizes sign language gestures

Tools and Languages

Programming Language

Python – Used for implementing the sign language recognition system and model inference.

Development Environment

Google Colab – Used for training the YOLOv8 model using GPU support.

Libraries and Frameworks

Ultralytics YOLOv8 – Used for training and deploying the sign language classification model.

OpenCV – Used for webcam access and real-time video frame processing.

NumPy – Used for numerical operations and data handling.

Dataset

ISL-CSLTR Dataset – Used for training and validating the sign language recognition model.

Tools

Visual Studio Code – Used for implementing the real-time recognition application.

References

- [1] A. Rastgoo, K. Kiani, and S. Escalera, “Sign Language Recognition: A Deep Survey,” IEEE Access, vol. 9, pp. 145812–145840, 2021.
- [2] S. Koller, O. Zargaran, H. Ney, and R. Bowden, “Deep Sign: Hybrid CNN-HMM for Continuous Sign Language Recognition,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 10, pp. 2593–2607, 2020.
- [3] J. Huang, W. Zhou, Q. Zhang, H. Li, and W. Li, “Video-Based Sign Language Recognition Without Temporal Segmentation,” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, no. 1, pp. 2257–2264, 2020.
- [4] T. Camgoz, O. Koller, S. Hadfield, and R. Bowden, “Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation,” IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10023–10033, 2020.
- [5] M. Almasre and H. Al-Nuaim, “A Real-Time Vision-Based Hand Gesture Recognition System Using Deep Learning,” IEEE Access, vol. 8, pp. 107233–107245, 2020.
- [6] S. Aly and A. Aly, “Deep Learning-Based Sign Language Recognition System for Static and Dynamic Gestures,” Journal of King Saud University – Computer and Information Sciences, vol. 34, no. 8, pp. 4214–4225, 2022.
- [7] M. Ko, J. Choi, and J. Kim, “Deep Learning-Based Hand Gesture Recognition Using CNN,” IEEE International Conference on Consumer Electronics, pp. 1–4, 2021.

References

- [8] A. Kumar, R. Sharma, and P. Singh, “Indian Sign Language Recognition Using Convolutional Neural Networks,” *Procedia Computer Science*, vol. 167, pp. 2003–2012, 2020.
- [9] M. Pigou, S. Dieleman, P. Kindermans, and B. Schrauwen, “Sign Language Recognition Using Convolutional Neural Networks,” *European Conference on Computer Vision Workshops*, pp. 572–578, 2020.
- [10] W. Shi, H. Wang, and X. Li, “Deep Learning-Based Continuous Sign Language Recognition Using Spatial-Temporal Networks,” *IEEE Access*, vol. 9, pp. 103977–103989, 2021.
- [11] H. Zhou, W. Zhou, and H. Li, “Spatial-Temporal Multi-Cue Network for Continuous Sign Language Recognition,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, pp. 13009–13016, 2020.
- [12] Y. Li, M. Rodriguez, and X. Zhang, “Vision-Based Sign Language Recognition Using Deep Neural Networks,” *IEEE International Conference on Image Processing (ICIP)*, pp. 3142–3146, 2022.
- [13] K. Li, Z. Zhang, and Q. Liu, “A Deep Neural Network Framework for Dynamic Two-Handed Indian Sign Language Recognition,” *Sensors (MDPI)*, vol. 25, no. 3652, 2025.
- [14] S. Tamura, T. Yanagi, and K. Kawamoto, “Hand Gesture Recognition Using YOLO-Based Object Detection and Deep Learning,” *IEEE International Conference on Artificial Intelligence and Computer Vision*, pp. 112–118, 2023.
- [15] P. Gupta and N. Sharma, “Real-Time Sign Language Recognition Using Deep Learning and Computer Vision Techniques,” *IEEE International Conference on Machine Learning and Applications*, pp. 589–594, 2024.

Thank You